

MEDALLION ARCHITECTURE · ETL

Loading the Silver Layer

How `silver.load_silver` cleanses, standardizes, and reshapes six Bronze tables from the CRM and ERP systems into trusted Silver tables.



`silver.load_silver` · T-SQL Stored Procedure



Pipeline Overview

Every table follows the same two-step pattern inside one TRY / CATCH block

1 TRUNCATE

Each Silver table is fully truncated before reload, so the layer always reflects the latest Bronze snapshot.

2 INSERT · TRANSFORM

Cleansed, standardized data is inserted — trimming text, mapping codes, fixing dates, and deduplicating rows.

3 LOG DURATION

Start/end timestamps are captured per table and printed, plus one total duration for the whole batch.

Six Bronze tables are loaded, in this order:

`crm_cust_info`

Customer master data

`crm_prd_info`

Product catalog

`crm_sales_details`

Sales transactions

`erp_cust_az12`

ERP customer demographics

`erp_loc_a101`

ERP customer location

`erp_px_cat_g1v2`

ERP product categories

1

crm_cust_info

Customer master data — CRM system

Bronze



Transform



Silver

SOURCE bronze.crm_cust_info

TARGET silver.crm_cust_info

Key Transformations

- 1 Deduplicate customers**
ROW_NUMBER() partitioned by cst_id, ordered by cst_create_date DESC — only the most recent record per customer survives.
- 2 Drop incomplete rows**
Rows with a NULL cst_id are filtered out before deduplication.
- 3 Trim text fields**
cst_firstname and cst_lastname are stripped of leading / trailing whitespace.
- 4 Standardize codes**
Marital status (S/M) and gender (F/M) codes are mapped to readable values; anything else becomes 'n/a'.

```
cst_marital_status  
'S' → 'Single'  
'M' → 'Married'  
NULL/other → 'n/a'
```

`SOURCE bronze.crm_prd_info``TARGET silver.crm_prd_info`

Key Transformations

1

Split the product key

First 5 characters (with '-' → '_') become `cat_id`; the remainder becomes the cleaned `prd_key`.

2

Default missing cost

NULL `prd_cost` values are replaced with 0 using `ISNULL()`.

3

Map product line codes

M/R/S/T are expanded to Mountain / Road / Other Sales / Touring; anything else becomes 'n/a'.

4

Derive the end date

`LEAD()` looks ahead to the next row's start date (per `prd_key`) and sets `prd_end_dt` to one day before it.

```
prd_key  
  'CO-RF-FR-R92B-58'  
→ cat_id  'CO_RF'  
→ prd_key 'FR-R92B-58'
```

SOURCE `bronze.crm_sales_details`

TARGET `silver.crm_sales_details`

Key Transformations

1

Validate integer dates

`sls_order_dt`, `sls_ship_dt`, `sls_due_dt` arrive as 8-digit integers; anything zero or not 8 digits is set to NULL, otherwise cast to DATE.

2

Recalculate sales

If `sls_sales` is missing, ≤ 0 , or doesn't equal $\text{quantity} \times |\text{price}|$, it's recomputed as $\text{sls_quantity} * \text{ABS}(\text{sls_price})$.

3

Derive unit price

If `sls_price` is missing or ≤ 0 , it's derived as $\text{sls_sales} / \text{sls_quantity}$ (guarded with NULLIF).

```
sls_sales check
  bad or mismatched
  → quantity × ABS(price)
  else kept as-is
```

SOURCE bronze.erp_cust_az12

TARGET silver.erp_cust_az12

Key Transformations

1

Strip 'NAS' prefix

Where cid begins with 'NAS', the prefix is removed so it can be joined to the CRM customer key.

2

Nullify future birthdates

Any bdate later than the current date is treated as bad data and set to NULL.

3

Standardize gender

'F' / 'FEMALE' → Female, 'M' / 'MALE' → Male; every other value becomes 'n/a'.

```
cid
'NASAW00011000'
→ 'AW00011000'
```

SOURCE `bronze.erp_loc_a101`

TARGET `silver.erp_loc_a101`

Key Transformations

1

Clean the customer key

Hyphens are removed from cid so it aligns with the customer key format used elsewhere in Silver.

2

Standardize country names

'DE' → Germany, 'US'/'USA' → United States, blank or NULL → 'n/a'; other values are trimmed and kept.

```
cntry
'US' / 'USA' → 'United States'
' ' / NULL → 'n/a'
```

SOURCE bronze.erp_px_cat_g1v2

TARGET silver.erp_px_cat_g1v2

Key Transformations



Direct pass-through

id, cat, subcat, and maintenance are already clean in Bronze, so they load into Silver unchanged.



Why include it at all?

Truncate-and-reload still applies — the table is refreshed every run to stay in sync with its Bronze source.

```
columns loaded
  id, cat, subcat,
  maintenance
(no CASE logic needed)
```


Error Handling & Logging

The entire load runs inside one TRY / CATCH block, with timing captured at every step



BEGIN TRY

- Records @batch_start_time at the very start of the run
- Loads all six tables in sequence: CRM tables first, then ERP tables
- Each table's TRUNCATE + INSERT is timed with @start_time / @end_time
- PRINT statements report each table's load duration in seconds
- @batch_end_time is captured once all tables finish loading
- A final PRINT reports total batch duration for the whole procedure



BEGIN CATCH

If any step fails, execution jumps here and the procedure prints three diagnostics instead of throwing an unhandled error:

ERROR_MESSAGE () the text of what went wrong

ERROR_NUMBER () the SQL Server error code

ERROR_STATE () the error state, for diagnostics

Note: the CATCH block logs the error but does not re-throw or roll back — downstream steps should check for it explicitly.

Result

A trusted Silver Layer

Six Bronze tables — cleansed, standardized, and deduplicated — ready for the Gold layer's business-ready views.

6

tables loaded

2

source systems — CRM & ERP

1

TRY/CATCH wrapped procedure

```
EXEC silver.load_silver;
```

